



Sistemas Embebidos I

Otoño 2026

Práctica #4: Interrupts(IRQ): latency&priorities

María Fernanda Conde Calatayud

Andrea Ceja Lara

31/08/26

Índice

<i>Práctica #4: Interrupts (IRQ): latency & priorities</i>	2
Introducción	2
Descripción de la práctica	2
Objetivo.....	2
Marco teórico.....	2
Desarrollo	4
Materiales.....	4
Esquemático.....	4
Desarrollo.....	5
Resultados	7
Evidencias	8
Conclusiones	8

Práctica #4: Interrupts (IRQ): latency & priorities

Introducción

Descripción de la práctica

La práctica consiste en configurar y medir el comportamiento de las interrupciones por hardware (IRQ) y la gestión de prioridades en el microcontrolador. Para ello, el desarrollo se divide en dos etapas:

1. Caracterización de una interrupción por GPIO:

Se configura un botón para generar una interrupción ante un flanco de entrada. Dentro de su función de servicio (ISR), se conmuta un pin de traza dedicado utilizando instrucciones de *SET/CLR* (subir a HIGH al entrar a la ISR y bajar a LOW al salir). Conectando un osciloscopio de doble canal (Canal 1 al botón y Canal 2 al pin de traza), se mide mediante los cursores la latencia de interrupción (tiempo entre el evento físico y el inicio de la ISR) y el tiempo de servicio (duración del pulso en alto).

2. Análisis de latencia bajo competencia de prioridades:

Se integra una segunda interrupción mediante una alarma de temporizador (*timer*) para generar competencia con el botón. Se mide la latencia del botón en tres escenarios: operando solo, compitiendo contra la alarma con menor prioridad y compitiendo contra la alarma con mayor prioridad. Finalmente, los cambios en los tiempos medidos se justifican analizando los mecanismos de *preemption*, *tail-chaining* y *late arrival* del NVIC.

Objetivo

Configurar y caracterizar el comportamiento de las interrupciones por hardware (GPIO y opcionalmente temporizador) en un microcontrolador, midiendo experimentalmente parámetros temporales clave mediante una técnica de traza doble en osciloscopio.

Marco teórico

Una interrupción es un evento por hardware o software que suspende temporalmente la ejecución del programa principal (*Background Code*) para atender de inmediato una función especializada denominada Rutina de Servicio de Interrupción (ISR, por sus siglas en inglés). Una vez finalizada la ISR, la unidad central de procesamiento (CPU) reanuda el código de fondo en el punto exacto donde fue interrumpido.

Para que una interrupción se active correctamente, deben habilitarse dos niveles de control o "candados":

1. El bit de permiso dentro del periférico específico que genera el evento.

2. La habilitación de la línea de interrupción dentro del controlador global (NVIC).

Cuando se produce una petición de interrupción (IRQ), la CPU guarda automáticamente en la pila un conjunto de registros de trabajo (R0-R3, R12, LR, PC, xPSR). Posteriormente, consulta la *Vector Table* para cargar la dirección de memoria de la ISR asociada. Al concluir la rutina, la CPU desapila los registros y retorna al hilo principal.

En la caracterización temporal de una interrupción se distinguen tres métricas principales:

- **Latencia de interrupción:** Tiempo transcurrido desde que la señal IRQ es válida hasta que se ejecuta la primera instrucción de la ISR. Incluye el tiempo de hardware consumido en el auto-apilado de registros y la búsqueda en la *Vector Table*.
- **Tiempo de servicio:** Duración total de la ejecución dentro de la ISR, contada desde su primera instrucción hasta la última. Se recomienda mantener el trabajo dentro de la ISR al mínimo indispensable y limpiar la fuente del evento lo antes posible.
- **Overhead de retorno:** Costo fijo asociado al desapilado de contexto y a la reanudación del código de fondo.

El controlador vectorial de interrupciones anidadas (NVIC) en arquitecturas ARM Cortex-M gestiona la urgencia de las peticiones. La asignación de prioridades sigue una escala inversa: a menor valor numérico, mayor es la urgencia del evento (siendo el nivel 0 la máxima prioridad).

En arquitecturas como Cortex-M33, se implementan habitualmente 4 bits para prioridades, lo que permite definir 16 niveles lógicos (0 a 15). Debido a que los registros del hardware son de 8 bits y únicamente utilizan los bits más significativos, los valores lógicos deben desplazarse hacia la izquierda antes de ser escritos. Por este motivo, funciones del SDK como `irq_set_priority()` requieren el ajuste de bits mediante fórmulas o macros de conversión como `NVIC_PRIO(level)`.

Cuando múltiples interrupciones ocurren de forma simultánea o cercana en el tiempo, el NVIC aplica los siguientes comportamientos:

- **Preemption:** Sucede cuando una ISR en ejecución es interrumpida por una nueva IRQ que posee un número de prioridad menor (más urgente).
- **Tail-chaining:** Ocurre cuando al finalizar una ISR existe otra IRQ pendiente de prioridad válida. El núcleo encadena directamente la ejecución de la siguiente ISR sin realizar el proceso completo de desapilado y apilado intermedio, reduciendo el *overhead* de transición.

- **Late arrival:** Se presenta si durante la fase de apilado de entrada a una ISR llega una segunda IRQ de mayor prioridad. El NVIC redirige la ejecución hacia la rutina más urgente antes de ejecutar la primera instrucción de la interrupción anterior.

Las interrupciones pueden ser configuradas en dos modalidades principales según la naturaleza de la señal:

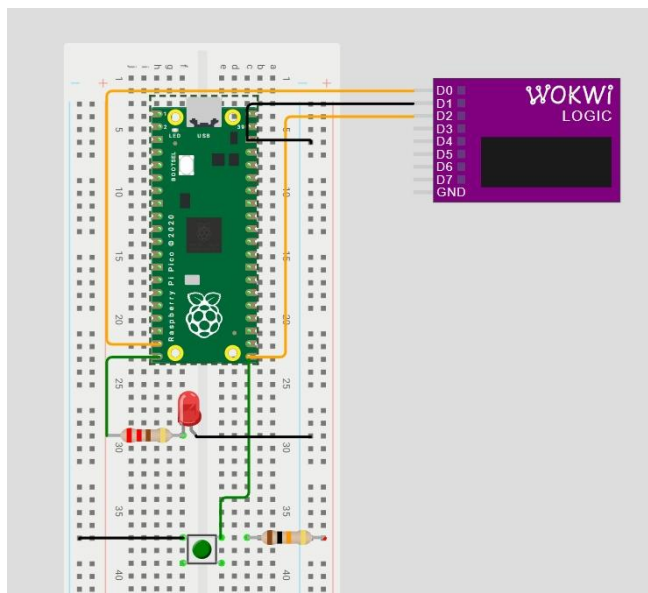
- **Por flanco (*Edge-triggered*):** Se activan en transiciones de estado lógico (flanco de subida RISING o flanco de bajada FALLING), siendo idóneas para eventos discretos como la pulsación de un botón.
- **Por nivel (*Level-triggered*):** Permanece pendiente mientras la condición lógica se mantenga activa (nivel alto HIGH o nivel bajo LOW). Requiere eliminar explícitamente la causa en el periférico para evitar que vuelva a reingresar de forma continua.

Desarrollo

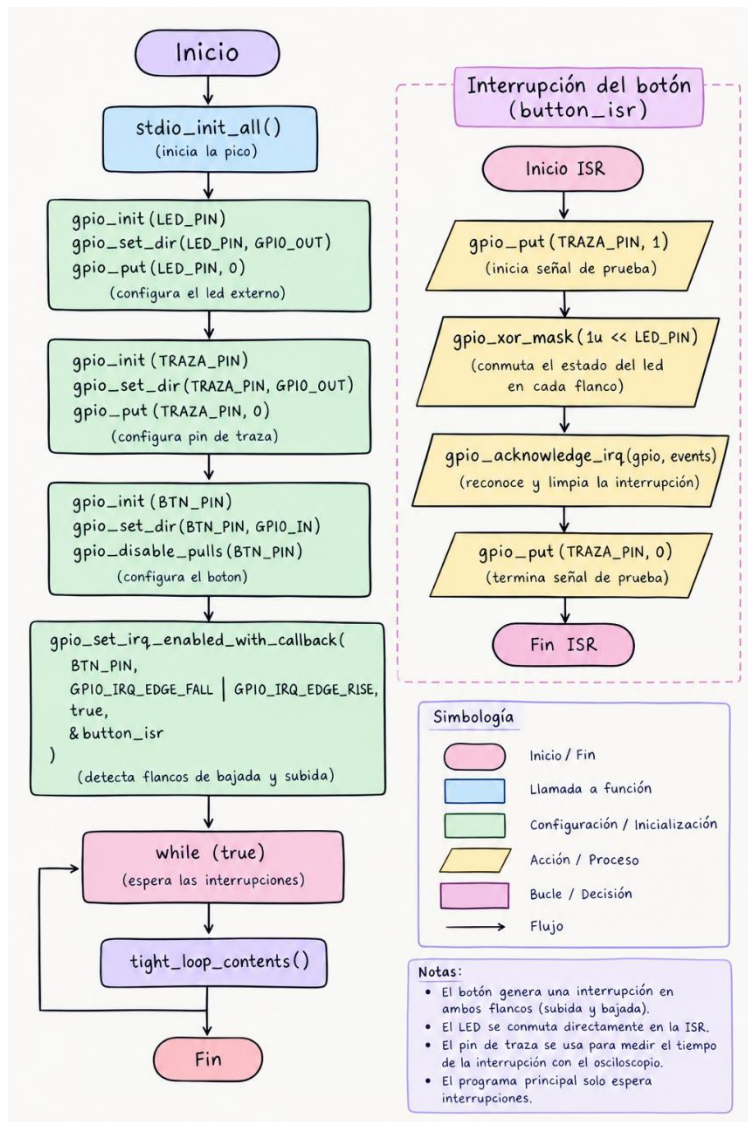
Materiales

- Raspberry Pi Pico
- Protoboard
- Jumpers
- Botón
- Diodo LED
- Resistencia 330 ohms
- Osciloscopio
- Simulador wowki

Esquemático



Desarrollo



```

1  #include "pico/stdlib.h"
2  #include "hardware/gpio.h"
3
4  #define LED_PIN 15    // Pin del led externo
5  #define BTN_PIN 16   // Pin del boton
6  #define TRAZA_PIN 14 // Pin de traza para osciloscopio
7
8  // funcion de interrupcion del boton
9  static void button_isr(uint gpio, uint32_t events) {
10     // inicia la señal de prueba
11     gpio_put(TRAZA_PIN, 1);
12
13     // conmuta el estado del led directamente en cada flanco detectado
14     gpio_xor_mask(1u << LED_PIN);
15
16     // reconoce y limpia la interrupcion que ya fue atendida
17     gpio_acknowledge_irq(gpio, events);
18
19     // termina la señal de prueba
20     gpio_put(TRAZA_PIN, 0);
21 }
22
23 int main(void) {
24     // inicia la pico
25     stdio_init_all();
26
27     // configura el led externo
28     gpio_init(LED_PIN);
29     gpio_set_dir(LED_PIN, GPIO_OUT);
30     gpio_put(LED_PIN, 0);
31
32     // configura el pin de prueba
33     gpio_init(TRAZA_PIN);
34     gpio_set_dir(TRAZA_PIN, GPIO_OUT);
35     gpio_put(TRAZA_PIN, 0);
36
37     // configura el boton
38     gpio_init(BTN_PIN);
39     gpio_set_dir(BTN_PIN, GPIO_IN);
40     gpio_disable_pulls(BTN_PIN);
41
42     // detecta tanto flanco de bajada como de subida para responder sin importar como cierre el boton en el circuito
43     gpio_set_irq_enabled_with_callback(
44         BTN_PIN,
45         GPIO_IRQ_EDGE_FALL | GPIO_IRQ_EDGE_RISE,
46         true,
47         &button_isr
48     );
49
50     // espera las interrupciones
51     while (true) {
52         tight_loop_contents();
53     }

```

Resultados

Cálculos independientes paso a paso

Tomando como referencia el primer evento en $t = 633\,344\,216\text{ ns}$:

1. Latencia de interrupción:

- El flanco del evento ocurre en $t_{\text{evento}} = 633\,344\,216\text{ ns}$.
- La traza de la ISR sube en $t_{\text{traza_sube}} = 633\,345\,424\text{ ns}$.

$$\Delta t_{\text{latencia}} = 633\,345\,424\text{ ns} - 633\,344\,216\text{ ns} = 1\,208\text{ ns} \approx 1.21\text{ }\mu\text{s}$$

2. Tiempo de servicio:

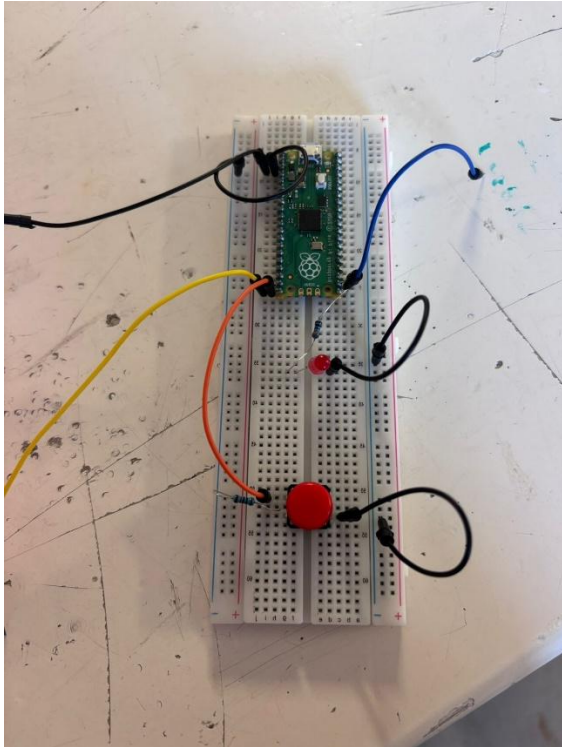
- La traza de la ISR sube en $t_{\text{traza_sube}} = 633\,345\,424\text{ ns}$.
- La traza de la ISR baja en $t_{\text{traza_baja}} = 633\,345\,600\text{ ns}$.

$$\Delta t_{\text{servicio}} = 633\,345\,600\text{ ns} - 633\,345\,424\text{ ns} = 176\text{ ns} \approx 0.18\text{ }\mu\text{s}$$

Tabla de resultados para el reporte

Parámetro	Valor esperado	Valor medido (osciloscopio / simulación)
Latencia de interrupción	Bajo, del orden de microsegundos	$1.21\text{ }\mu\text{s}$ (1 208 ns)
Tiempo de servicio	Depende del trabajo dentro de la ISR	$0.18\text{ }\mu\text{s}$ (176 ns)
Osciloscopio	—	Canal 1: Evento (Botón / D2) Canal 2: Traza (ISR / D0)

Evidencias



Conclusiones

Esta práctica permite caracterizar y medir el comportamiento real de las interrupciones (IRQ) en un sistema embebido bare-metal mediante el uso de un osciloscopio y una técnica de doble traza. Al monitorear el evento físico en un canal y un pin de traza configurado con instrucciones directas SET y CLR en el otro, es posible calcular de forma objetiva tanto la latencia de respuesta del hardware como el tiempo de servicio de la rutina de atención. Asimismo, el ejercicio demuestra la importancia de configurar correctamente los niveles de prioridad en el NVIC para controlar el comportamiento del sistema ante eventos simultáneos o de mayor urgencia, observando de forma práctica fenómenos clave como *preemption*, *tail-chaining* y *late arrival*. En conclusión, la práctica reafirma las reglas fundamentales de diseño en sistemas embebidos: limpiar la fuente de la interrupción oportunamente y mantener las ISRs con una carga de trabajo mínima para reducir la latencia global y garantizar un sistema reactivo y determinista.

Referencias

[1] M. N. Ramírez Cruz, "Bare-Metal I/O & Timing: Interrupts (IRQ): latency & priorities," presentación de la asignatura *Sistemas Embebidos*, Universidad Iberoamericana Puebla, Puebla, México, Otoño 2026.